ASSIGNMENT-12.4

Name:R.SHASHIDEEPIKA

HT.NO:2303A51871

Batch:13

Task 1: Bubble Sort for Ranking Exam

Scores

Scenario

You are working on a college result processing system where a small

list of student scores needs to be sorted after every internal assessment.

Task Description

• Implement Bubble Sort in Python to sort a list of student scores.

• Use an AI tool to:

o Insert inline comments explaining key operations such as

comparisons, swaps, and iteration passes

o Identify early-termination conditions when the list

becomes sorted

o Provide a brief time complexity analysis

Expected Outcome

• A Bubble Sort implementation with:

o AI-generated comments explaining the logic

o Clear explanation of best, average, and worst-case

complexity

o Sample input/output showing sorted scores

Write a Python program to sort student exam scores using Bubble Sort with inline comments explaining comparisons, swaps, and passes, include early termination

optimization, add best/average/worst time complexity analysis, and show sample input and output.

```python
bubble sort t1.py > ...
#Write a Python program to sort student exam scores using Bubble Sort with inline comments explaining comparisons, swaps, and passes, include early termination optimizati
def bubble_sort(scores):
    n = len(scores)
    # Flag to check if any swapping occurs
    swapped = True

    # Outer loop to traverse through all elements
    for i in range(n):
        # If no swapping occurs, the array is already sorted
        if not swapped:
            break

        # Reset the flag for this pass
        swapped = False

        # Inner loop to compare adjacent elements
        for j in range(0, n-i-1):
            # Compare the current element with the next element
            if scores[j] > scores[j+1]:
                # Swap if the current element is greater than the next element
                scores[j], scores[j+1] = scores[j+1], scores[j]
                swapped = True  # Set the flag to True if a swap occurred

    return scores
# Sample input: List of student exam scores
scores = [88, 75, 92, 85, 79]
# Sorting the scores using bubble sort
sorted_scores = bubble_sort(scores)
# Sample output: Sorted list of student exam scores
print("Sorted Scores:", sorted_scores)
# Time Complexity Analysis:
# Best Case: O(n) - This occurs when the input list is already sorted, and the algorithm makes one pass to check that it is sorted without performing any swaps.
# Average Case: O(n^2) - This occurs when the input list is in random order, and the algorithm needs to perform multiple passes and swaps to sort the list.
# Worst Case: O(n^2) - This occurs when the input list is sorted in reverse order, and the algorithm needs to perform the maximum number of comparisons and swaps to sort
# Sample Input: [88, 75, 92, 85, 79]
# Sample Output: Sorted Scores: [75, 79, 85, 88, 92]
```

```
PS C:\Users\bhuky\Desktop\AI AC> & C:/Users/bhuky/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/bhuky/Desktop/AI AC/bubble sort t1.py"
Sorted Scores: [75, 79, 85, 88, 92]
PS C:\Users\bhuky\Desktop\AI AC>
```

Justification :

The program correctly implements the Bubble Sort algorithm using nested loops to compare and swap adjacent elements. It includes clear inline comments explaining comparisons, swaps, and iteration passes. The early termination optimization using the swapped flag improves efficiency when the list is already sorted. Additionally, the time complexity analysis and sample input/output are properly provided. Therefore, the solution meets all task requirements.

Task 2: Improving Sorting for Nearly Sorted

Attendance Records

Scenario

You are maintaining an attendance system where student roll numbers

are already almost sorted, with only a few late updates.

Task Description

• Start with a Bubble Sort implementation.

• Ask AI to:

o Review the problem and suggest a more suitable sorting

algorithm

o Generate an Insertion Sort implementation

o Explain why Insertion Sort performs better on nearly

sorted data

• Compare execution behavior on nearly sorted input

Expected Outcome

• Two sorting implementations:

o Bubble Sort

o Insertion Sort

• AI-assisted explanation highlighting efficiency differences for

partially sorted datasets

Write a Python program implementing both Bubble Sort and Insertion Sort for nearly sorted attendance roll numbers, compare their performance, and explain why Insertion Sort is more efficient for partially sorted data.

#Write a Python program implementing bot.py > ...

```python
#Write a Python program implementing both Bubble Sort and Insertion Sort for nearly sorted attendance roll numbers, compare their performance, and explain why Insertion S
import time
def bubble_sort(arr):
    n = len(arr)
    swapped = True

    for i in range(n):
        if not swapped:
            break
        swapped = False

        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
    return arr

def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
# Sample input: Nearly sorted attendance roll numbers
roll_numbers = [101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115]
# Introducing a few unsorted elements
roll_numbers[3] = 110  # Swapping elements to make it nearly sorted
roll_numbers[7] = 105
# Timing Bubble Sort
start_time = time.time()
sorted_bubble = bubble_sort(roll_numbers.copy())
end_time = time.time()
print("Bubble Sort Result:", sorted_bubble)
```

```python
end_time = time.time()
print("Bubble Sort Result:", sorted_bubble)
print("Bubble Sort Time:", end_time - start_time)

# Timing Insertion Sort
start_time = time.time()
sorted_insertion = insertion_sort(roll_numbers.copy())
end_time = time.time()
print("Insertion Sort Result:", sorted_insertion)
print("Insertion Sort Time:", end_time - start_time)
# Explanation:
# Insertion Sort is more efficient for partially sorted data because it builds the sorted array one item at a time, and it can quickly place elements in their correct pos
# Time Complexity Analysis:
# Bubble Sort: O(n^2) in worst and average cases, O(n) in best case (already sorted)
# Insertion Sort: O(n^2) in worst and average cases, O(n) in best case (already sorted)
# Sample Input: [101, 102, 110, 104, 105, 106, 107, 105, 109, 110, 111, 112, 113, 114, 115]
# Sample Output:
# Bubble Sort Result: [101, 102, 104, 105, 105, 106, 107, 109, 110, 110, 111, 112, 113, 114, 115]
# Bubble Sort Time: (time taken for bubble sort)
# Insertion Sort Result: [101, 102, 104, 105, 105, 106, 107, 109, 110, 110, 111, 112, 113, 114, 115]
# Insertion Sort Time: (time taken for insertion sort)
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                                    Python
PS C:\Users\bhuky\Desktop\AI AC> & C:/Users/bhuky/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/bhuky/Desktop/AI AC/#Write a Python program implementing bot.py"
Bubble Sort Result: [101, 102, 103, 105, 105, 106, 107, 109, 110, 110, 111, 112, 113, 114, 115]
Bubble Sort Time: 0.0
Insertion Sort Result: [101, 102, 103, 105, 105, 106, 107, 109, 110, 110, 111, 112, 113, 114, 115]
Insertion Sort Time: 0.0
PS C:\Users\bhuky\Desktop\AI AC>
```

Justification :

The program effectively compares two sorting algorithms on nearly sorted input to analyze their behavior. Bubble Sort may still require multiple passes, even if only one element is out of place. In contrast, Insertion Sort quickly adjusts small displacements with minimal shifting. Its best-case time complexity of O(n) makes it ideal for such datasets. Hence, the implementation and explanation accurately highlight the efficiency differences.

Task 3: Searching Student Records in a Database

Scenario

You are developing a student information portal where users search for

student records by roll number.

Task Description

• Implement:

o Linear Search for unsorted student data

o Binary Search for sorted student data

• Use AI to:

o Add docstrings explaining parameters and return values

o Explain when Binary Search is applicable

o Highlight performance differences between the two

searches

Expected Outcome

• Two working search implementations with docstrings

• AI-generated explanation of:

o Time complexity

o Use cases for Linear vs Binary Search

• A short student observation comparing results on sorted vs

unsorted lists’

Write a Python program implementing Linear and Binary Search for student roll numbers with docstrings, time complexity explanation, use cases, and performance comparison.

#Write a Python program implementing Lin.py > ...

```python
# Write a Python program implementing Linear and Binary Search for student roll numbers
# with docstrings, time complexity explanation, use cases, and performance comparison.

import time


def linear_search(arr, target):
    """
    Perform a Linear Search for the target in the given array.

    Time Complexity:
    - Best Case: O(1)  -> Target found at first position
    - Worst Case: O(n) -> Target found at last position or not present

    Use Cases:
    - Suitable for small or unsorted datasets.
    - Does not require sorted data.

    Parameters:
    arr (list): The list of student roll numbers.
    target (int): The roll number to search for.

    Returns:
    int: Index of the target if found, otherwise -1.
    """
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1


def binary_search(arr, target):
    """
    Perform a Binary Search for the target in a sorted array.

    Time Complexity:
    - Best Case: O(1)      -> Target found at middle
```

```python
    Time Complexity:
    - Best Case: O(1)      -> Target found at middle
    - Worst Case: O(log n) -> Search space halves each step

    Use Cases:
    - Suitable for large sorted datasets.
    - Requires the array to be sorted before searching.

    Parameters:
    arr (list): The sorted list of student roll numbers.
    target (int): The roll number to search for.

    Returns:
    int: Index of the target if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1


def performance_comparison(arr, target):
    """
    Compare the performance of Linear Search and Binary Search.

    Parameters:
```

```python
    # Measure Linear Search time
    start_time = time.time()
    linear_result = linear_search(arr, target)
    linear_time = time.time() - start_time

    # Binary Search requires sorted data
    sorted_arr = sorted(arr)

    # Measure Binary Search time
    start_time = time.time()
    binary_result = binary_search(sorted_arr, target)
    binary_time = time.time() - start_time

    print("\nSearching for Roll Number:", target)
    print("Linear Search -> Index:", linear_result, "| Time:", f"{linear_time:.6f} seconds")
    print("Binary Search -> Index:", binary_result, "| Time:", f"{binary_time:.6f} seconds")


# Example Usage
if __name__ == "__main__":

    # Sorted data
    student_roll_numbers = [101, 102, 103, 104, 105, 106, 107, 108, 109, 110]
    target_roll_number = 105

    print("----- Using Sorted Data -----")
    performance_comparison(student_roll_numbers, target_roll_number)

    # Unsorted data
    unsorted_roll_numbers = [110, 101, 104, 103, 108, 102, 107, 105, 109, 106]

    print("\n----- Using Unsorted Data -----")
    performance_comparison(unsorted_roll_numbers, target_roll_number)
```

```
PS C:\Users\bhuky\Desktop\AI AC> & C:/Users/bhuky/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/bhuky/Desktop/AI AC/#Write a Python program implementing Lin.py"
----- Using Sorted Data -----

Searching for Roll Number: 105
Linear Search -> Index: 4 | Time: 0.000000 seconds
Binary Search -> Index: 4 | Time: 0.000000 seconds

----- Using Unsorted Data -----

Searching for Roll Number: 105
Linear Search -> Index: 7 | Time: 0.000000 seconds
Binary Search -> Index: 4 | Time: 0.000000 seconds
PS C:\Users\bhuky\Desktop\AI AC>
```

Justification :

The program correctly implements both Linear Search and Binary Search with proper docstrings explaining parameters, return values, time complexity, and use cases. Linear Search works on both sorted and unsorted data, making it flexible but less efficient for large datasets. Binary Search, however, significantly reduces search time by dividing the

search space in half, but it requires sorted data. The performance comparison clearly shows that Binary Search is faster for sorted lists. This demonstrates a clear understanding of when to use each algorithm.

Task 4: Choosing Between Quick Sort and Merge Sort for Data Processing

Scenario

You are part of a data analytics team that needs to sort large datasets

received from different sources (random order, already sorted, and

reverse sorted).

Task Description

• Provide AI with partially written recursive functions for:

o Quick Sort

o Merge Sort

o**• Ask AI to:**

Complete the recursive logic

o Add meaningful docstrings

o Explain how recursion works in each algorithm

• Test both algorithms on:

o Random data

o Sorted data

o Reverse-sorted data

Expected Outcome

• Fully functional Quick Sort and Merge Sort implementations

• AI-generated comparison covering:

o Best, average, and worst-case complexities

o Practical scenarios where one algorithm is preferred over

the other

Task 5: Optimizing a Duplicate Detection Algorithm

Scenario

You are building a data validation module that must detect duplicate

user IDs in a large dataset before importing it into a system.

Task Description

- Write a naive duplicate detection algorithm using nested loops.
- Use AI to:
  - o Analyze the time complexity
  - o Suggest an optimized approach using sets or dictionaries
  - o Rewrite the algorithm with improved efficiency
- Compare execution behavior conceptually for large input sizes

Expected Outcome

- Two versions of the algorithm:
  - o Brute-force ($O(n^2)$)
  - o Optimized ($O(n)$)